



**Prince Mohammad Bin Fahd University College of Computer
Engineering and Science.**

**Secure Software Engineering
Course Project**

Report

Online Secure Student Information System Spring 2025 - 2026

Group Members:

Julan Almuhawesh - 202201147

Reyam Alhammadi - 202201409

Muneerah Alsaeed - 202101182

Table of Contents

1.	Introduction	3
1.1	Overview	3
1.2	Objective.....	3
1.3	Scope	3
2.	Security Requirements Analysis	3
2.1	Functional and Non-functional Requirements.....	3
2.1.1	Functional Requirements.....	3
2.1.2	Non-functional Requirements	4
2.2	Identified Security Requirements	4
2.3	Identified Assets	5
2.4	Identified Threats.....	5
2.5	Risk Assessment and Management (Risk Matrix)	6
2.6	Identified Misuse Cases.....	6
3.	Securing Design	7
3.1	System Architecture and Components (Security Focus).....	7
3.2	Threat Modeling (Microsoft's STRIDE Approach).....	7
3.3	Security Design Principles.....	8
4.	Secure Implementation.....	9
4.1	Secure Coding Guidelines	9
4.2	Defensive Code for the Login Process	11
5.	Secure Testing.....	12
5.1	Security Test Cases for Identified Threats	12
5.2	Security Test Cases for Each Function in the System.....	14
6.	Conclusion.....	15
7.	References	16

1. Introduction

1.1 Overview

This report details the design and implementation of an Online Secure Student Information System (SIS), developed as a course project for Secure Software Engineering. The primary goal of this project is to design a secure software system by integrating security principles and best practices throughout the Software Development Life Cycle (SDLC). The system aims to provide a robust and secure platform for managing student grades, allowing students to view their academic performance and instructors to efficiently post and update grades.

1.2 Objective

The primary goal of this project is to design a secure software system by integrating security principles and best practices throughout the Software Development Life Cycle (SDLC).

1.3 Scope

The project scope encompasses the development of a web-based application that supports essential functionalities such as user account creation, account updates, secure login, viewing of grades by students, and posting/updating of grades by instructors. A significant emphasis is placed on the security aspects of the system, particularly in the login process, to safeguard sensitive academic data and user credentials.

2. Security Requirements Analysis

2.1 Functional and Non-functional Requirements

2.1.1 Functional Requirements

- **User Authentication:** The system must provide a secure login mechanism for both students and instructors.
- **Account Management:** Users (students and instructors) must be able to create and update their accounts.
- **Grade Viewing:** Students must be able to view their assigned grades for enrolled courses.
- **Grade Management:** Instructors must be able to post new grades and update existing grades for students in their courses.
- **Role-Based Access Control (RBAC):** The system must differentiate user privileges based on their roles (Student or Instructor), ensuring that only authorized actions can be performed.

2.1.2 Non-functional Requirements

- **Confidentiality:** All sensitive data, especially student grades and personal information, must be protected from unauthorized disclosure.
- **Integrity:** The accuracy and completeness of grades and user data must be maintained, preventing unauthorized modification or corruption.
- **Availability:** The system must be accessible to legitimate users when required, ensuring reliable operation.
- **Authenticity:** The identity of all users (students and instructors) must be verified during authentication to prevent impersonation.
- **Non-repudiation:** Actions performed within the system, such as an instructor posting a grade, must be provable and undeniable.
- **Security:** The system must be resilient against common web application vulnerabilities, including but not limited to SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).

2.2 Identified Security Requirements

To meet the functional and non-functional security requirements, the following specific security measures are mandated:

- **Secure Authentication:** Implementation of strong password policies, robust password hashing algorithms (e.g., BCrypt), and secure session management to protect user credentials.
- **Authorization Enforcement:** Strict enforcement of RBAC to ensure that users can only access resources and perform operations consistent with their assigned roles.
- **Data Protection:** Encryption of sensitive data both at rest (in the database) and in transit (using HTTPS/TLS for all network communications).
- **Input Validation:** Comprehensive validation of all user inputs on both client and server sides to prevent various injection attacks and data integrity issues.
- **Secure Error Handling:** Implementation of error handling mechanisms that provide generic messages to users while logging detailed error information internally, avoiding information leakage.
- **Logging and Monitoring:** Establishment of a system for logging security-relevant events, enabling auditing, forensic analysis, and timely incident response.

2.3 Identified Assets

The critical assets within the Secure SIS are categorized and valued as follows:

Asset Category	Specific Assets	Value (High/Medium/Low)
Data	Student Grades	High
	Student Personal Information (Name, ID, Email)	High
	Instructor Personal Information (Name, Email)	Medium
	User Credentials (Hashed Passwords)	High
Software	Web Application Codebase	High
	Database Schema and Data	High
Hardware/Infra	Web Server	Medium
	Database Server	Medium
	Network Infrastructure	Medium

Table 1: Identified Assets categorized

2.4 Identified Threats

Potential threats to the Secure SIS have been identified and categorized:

Threat Category	Specific Threats	Impact (High/Medium/Low)
Injection Attacks	SQL Injection (manipulating database queries)	High
	Cross-Site Scripting (XSS) (injecting client-side scripts)	Medium
Authentication/ Auth	Brute-force attacks on login credentials	High
	Session hijacking (taking over an authenticated session)	High
	Broken Authentication/Session Management	High
	Insecure Direct Object References (IDOR)	Medium

Data Exposure	Sensitive Data Exposure (unencrypted sensitive data)	High
Denial of Service	DoS/DDoS attacks (rendering the system unavailable)	Medium
Malicious Code	Malware/Viruses (compromising system integrity)	Medium
Configuration	Security Misconfiguration (weak default settings)	Medium

Table 2: Identified Threats

2.5 Risk Assessment and Management (Risk Matrix)

Risks are assessed using a matrix that combines the likelihood of a threat occurring with its potential impact. This allows for prioritization of mitigation efforts.

Likelihood \ Impact	Low	Medium	High
High	Medium	High	Critical
Medium	Low	Medium	High
Low	Low	Low	Medium

Table 3: Risk Matrix

Examples of Risks and Mitigation Strategies:

- **SQL Injection (Critical Risk):** Due to high likelihood and high impact, this is a critical risk. Mitigation involves using parameterized queries, prepared statements, and rigorous input validation.
- **Brute-force Login (Critical Risk):** High likelihood and high impact. Mitigation includes implementing account lockout policies after several failed attempts, CAPTCHA challenges, and rate limiting login attempts.
- **Sensitive Data Exposure (High Risk):** Medium likelihood but high impact. Mitigation strategies include encrypting sensitive data at rest and in transit, and enforcing strict access controls.

2.6 Identified Misuse Cases

Misuse cases describe scenarios where an attacker or unauthorized user attempts to exploit vulnerabilities or misuse system functionalities:

- **Unauthorized Grade Modification:** An instructor attempts to alter a grade for a student not enrolled in their course, or a student attempts to modify their own grade.
- **Unauthorized Data Access:** A student attempts to view another student's grades or an instructor's personal information without proper authorization.

- **Account Takeover:** An attacker gains unauthorized access to a legitimate user's account through various means, such as credential stuffing, phishing, or exploiting weak authentication.
- **System Sabotage:** An attacker attempts to disrupt the system's operation, corrupt data, or render the service unavailable to legitimate users.

3. Securing Design

3.1 System Architecture and Components (Security Focus)

The Secure SIS adopts a **three-tier architecture** to ensure a clear separation of concerns, enhance scalability, and improve maintainability, with security considerations integrated into each layer:

- **Presentation Layer (Client-side):** This layer handles user interaction and data display. Security is addressed through client-side input validation to provide immediate feedback and reduce server load, and by ensuring all communications with the server are encrypted via HTTPS/TLS.
- **Application Layer (Server-side / Business Logic):** Developed using Java Servlets and JSP, this layer contains the core business logic. It includes robust security modules for authentication and authorization, API rate limiting, and secure session management (e.g., using secure HTTP sessions with timeout management). All business logic is designed with security principles in mind, such as input sanitization and secure data handling.
- **Data Access Layer (Persistence Layer):** This layer manages interactions with the Apache Derby database. Security measures include using JDBC with prepared statements to prevent SQL injection, implementing the principle of least privilege for database user accounts, and considering encryption for sensitive data at rest within the database.

3.2 Threat Modeling (Microsoft's STRIDE Approach)

The STRIDE threat model has been applied to systematically identify and categorize potential threats and their corresponding mitigation strategies:

- **Spoofing:** Threats involving impersonation of legitimate users or system components. Mitigation includes strong authentication mechanisms, such as multi-factor authentication where applicable, and robust identity verification processes.

- **Tampering:** Threats related to unauthorized modification of data or system configurations. Mitigation strategies involve data integrity checks, digital signatures, and stringent access control policies.
- **Repudiation:** Threats where an entity denies having performed an action. Mitigation focuses on implementing comprehensive audit trails, detailed logging of security-relevant events, and non-repudiation mechanisms.
- **Information Disclosure:** Threats concerning the unauthorized exposure of sensitive data. Mitigation includes encryption of data at rest and in transit, strict access controls, and secure error handling to prevent information leakage.
- **Denial of Service (DoS):** Threats aimed at making system resources unavailable to legitimate users. Mitigation involves implementing rate limiting, load balancing, and designing a resilient infrastructure capable of handling high traffic and mitigating attack vectors.
- **Elevation of Privilege:** Threats where an attacker gains unauthorized higher-level access than initially granted. Mitigation strategies include enforcing the principle of least privilege, robust Role-Based Access Control (RBAC), and secure system configuration.

3.3 Security Design Principles

The following security design principles are fundamental to the Secure SIS:

- **Principle of Least Privilege:** Every user, process, and program should operate with the minimum set of privileges necessary to complete its task. This limits the potential damage from a compromise.
- **Defense in Depth:** Employing multiple layers of security controls (e.g., network, host, application, data) so that if one layer is breached, others remain to protect the system.
- **Secure Defaults:** All system components and configurations are set to the most secure options by default, requiring explicit action to reduce security.
- **Fail Securely:** In the event of a system failure or error, the system should default to a secure state, typically by denying access or ceasing operation rather than exposing vulnerabilities.
- **Separation of Duties:** Critical tasks divided among multiple individuals or roles to prevent a single point of failure or malicious activity by one person.

- **Input Validation:** All inputs received from users or external systems rigorously validated against expected formats, types, and ranges to prevent injection attacks (e.g., SQL Injection, XSS) and other data manipulation attempts.
- **Error Handling:** Generic error messages displayed to users to prevent information leakage, while detailed error logs maintained internally for debugging and security monitoring.
- **Session Management:** Secure session IDs generated and managed, with appropriate session timeouts and re-authentication requirements for sensitive operations to prevent session hijacking.
- **Database Security:** Utilization of parameterized queries and prepared statements for all database interactions, encryption of sensitive data fields, and secure configuration of database connections and user permissions.
- **Website Security:** Implementation of protections against common web vulnerabilities as outlined by the OWASP Top 10, including measures against Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and broken access control.
- **Encryption Techniques:** Strong cryptographic algorithms used for password hashing (e.g., BCrypt) and for securing data in transit (TLS/HTTPS). Consideration for AES encryption for sensitive data at rest if deemed necessary.

4. Secure Implementation

4.1 Secure Coding Guidelines

Adherence to secure coding guidelines is paramount to building a robust and secure application. Key guidelines include:

- **Input Validation and Sanitization:** All user inputs must be validated against expected patterns and sanitized to remove or neutralize malicious content. This prevents injection attacks (SQL, XSS, Command Injection).
- **Server-Side Validation:** All sensitive user inputs such as username, email, password, and role fields are validated on the server side before processing. Validation rules ensure that malicious or malformed data cannot bypass client-side validation controls.

- **Username Validation:** Usernames are restricted to 9–20 characters and may only contain letters, numbers, and underscores. This prevents invalid or malicious input patterns.

The image shows a web form titled "Secure SIS System". At the top, there is a blue header bar with the title in white. Below the header, a light red box contains a validation message: "Username must be 9-20 characters and contain only letters, numbers, or underscores." Below this message are two buttons: "Login" (blue text on a white button) and "Register" (green text on a white button). Under the buttons are two input fields: the first is labeled "username" and the second is a password field represented by a series of dots. The entire form is set against a light gray background.

- **Email Validation:** Email addresses are validated using regular expression (regex) patterns to ensure proper formatting and prevent invalid entries.
- **Password Strength Validation:** Passwords must satisfy strong security requirements, including:
 - Minimum length of 8 characters
 - At least one uppercase letter
 - At least one lowercase letter
 - At least one number
 - At least one special character

This reduces the risk of weak passwords and brute-force attacks.

- **Duplicate Username and Email Prevention:** Before creating a new account, the system checks whether the username or email already exists in the database. This prevents duplicate accounts and improves account integrity.
- **Role-Based Authorization Checks:** Authorization checks are enforced on the server side to ensure that only users with the appropriate role can access protected resources or perform privileged actions. For example, only instructors can manage grades, while students can only view their own grades.
- **Secure Exception Handling:** Instead of using `printStackTrace()`, the system uses secure logging mechanisms and generic user-facing error messages. Detailed exception information is recorded internally for debugging and auditing purposes while preventing sensitive system information leakage to users.
- **Parameterized Queries:** Always use prepared statements or parameterized queries for database interactions to prevent SQL Injection. Never concatenate user input directly into SQL queries.

- **Strong Password Hashing:** Store passwords using strong, one-way hashing algorithms like BCrypt or Argon2, along with a unique salt for each password. Never store passwords in plain text.
- **Secure Session Management:** Generate strong, random session IDs. Implement appropriate session timeouts and invalidate sessions upon logout or inactivity. Use secure cookies (HttpOnly, Secure flags).

- **Error Handling:** Implement custom error pages and log detailed error messages internally. Avoid displaying sensitive system information or stack traces to end-users.
- **Access Control:** Implement Role-Based Access Control (RBAC) at both the UI and API levels. Ensure that authorization checks are performed on the server-side for every request accessing protected resources.
- **Cross-Site Scripting (XSS) Prevention:** Encode all output that includes user-supplied data before rendering it in HTML to prevent XSS attacks.
- **Cross-Site Request Forgery (CSRF) Prevention:** Implement CSRF tokens for all state-changing requests to ensure that requests originate from the legitimate application.
- **Secure Configuration:** Ensure that all application and server configurations follow security best practices. Disable unnecessary services, remove default credentials, and keep software updated.
- **Logging:** Implement comprehensive logging of security-relevant events, including authentication attempts, authorization failures, and critical system actions. Ensure logs are protected from tampering.

4.2 Defensive Code for the Login Process

The login process is a critical entry point and requires robust defensive coding. The provided implementation focuses on:

- **Input Validation:** The login form will validate that both username and password fields are not empty. Further validation can be added for username format (e.g., email format).
- **Password Hashing:** The BCrypt library is used to hash and verify passwords securely. When a user registers, their password is hashed before storage. During login, the provided password is verified against the stored BCrypt hash.
- **Enhanced Login Security Controls.**
 - Server-side credential validation is performed during authentication to verify that all submitted login data meets expected formats and security requirements.
 - The authentication system validates password strength during account registration to enforce strong password policies before credentials are stored.
 - User passwords are securely hashed using BCrypt with salting before being stored in the database. During login, the submitted password is verified against the stored BCrypt hash using secure comparison methods.

- The system prevents unauthorized access through role verification and session-based authorization checks after successful authentication.
- Exception handling during the login process uses secure logging methods rather than directly exposing stack traces or sensitive debugging information.
- **SQL Injection Prevention:** By using JDBC PreparedStatement queries, all database interactions for user authentication securely handle user input through parameterized queries, effectively preventing SQL injection vulnerabilities.
 - **Authentication Mechanism:** The authentication flow is handled using Java Servlets, secure session management, and BCrypt password verification, which includes:
 - Intercepting login requests.
 - Loading user details through custom authentication methods implemented in AuthServicelet.
 - Comparing provided credentials with stored hashed passwords.
 - Managing authenticated sessions.

The image shows a web form for a 'Secure SIS System'. At the top, a green box displays 'Login successful'. Below this, there are two buttons: 'Login' (blue) and 'Register' (green). Underneath the buttons are two input fields: the first contains the text 'student1' and the second is masked with dots. At the bottom is a large blue 'Login' button.

- **Error Handling:** Generic error messages are displayed for invalid credentials (e.g., "Invalid username or password") without revealing specific details about why the login failed (e.g., whether the username exists or not).
- **Account Lockout/Rate Limiting:** While not fully implemented in the provided login code, the security configuration allows for integration of mechanisms like account lockout after multiple failed login attempts and rate limiting to prevent brute-force

attacks.

5. Secure Testing

5.1 Security Test Cases for Identified Threats

For each identified critical or high-impact threat, specific security test cases will be developed to verify the effectiveness of the implemented countermeasures.

- **Threat: SQL Injection**
 - **Test Case 1:** Attempt to log in with a username containing SQL injection payloads (e.g., 'OR '1'='1'). **Expected result:** Login fails, and no unauthorized access or database manipulation occurs. The application should handle the input safely using prepared statements.

The screenshot displays a web application interface titled "Secure SIS System". At the top, a blue header bar contains the title. Below the header, a light red error message box states "Invalid username or password". Underneath this, there are two buttons: "Login" (blue text on a white background) and "Register" (green text on a white background). Below the buttons, there are two input fields. The first input field contains the SQL injection payload "' OR '1'='1'". The second input field contains a masked password represented by seven dots. At the bottom of the form, there is a large blue button labeled "Login".

- **Threat: Brute-force Login**

- **Test Case 1:** Attempt multiple (e.g., 100) login attempts with invalid credentials for a valid username within a short period.

Expected result: The system should implement rate limiting or account lockout after a predefined number of failed attempts, preventing further login attempts for a specified duration.

- **Threat: Sensitive Data Exposure (e.g., passwords in logs)**

- **Test Case 1:** Verify that passwords are not logged in plain text in application logs during authentication or any other process.

Expected result: Passwords are only stored and processed as hashes, and never appear in logs.

- **Threat: Cross-Site Scripting (XSS)**

- **Test Case 1:** Attempt to inject a simple XSS payload (e.g., <script>alert('XSS');</script>) into user input fields (e.g., username, if it were displayed back).

Expected result: The script should not execute; the output should be properly encoded or sanitized.

- **Threat: Weak Password Usage**

- **Test Case:** Attempt to register using weak passwords such as "123456" or "password".
- **Expected Result:** Registration is rejected, and the system displays a password strength validation error.

- **Threat: Duplicate Account Creation**

- **Test Case:** Attempt to register with an existing username or email address.
- **Expected Result:** Registration fails, and the user is informed that the username or email already exists.

- **Threat: Unauthorized Role Access**

- **Test Case:** A student attempts to directly access instructor dashboard resources through URL manipulation.
- **Expected Result:** Access is denied and the user receives an authorization error or redirection response.

5.2 Security Test Cases for Each Function in the System

Beyond threat-specific tests, each core function of the system will have security test cases to ensure proper authorization and data handling.

- **Function: User Login**

- **Test Case 1:** Verify that a user with valid credentials for an 'INSTRUCTOR' role can successfully log in and is redirected to the instructor dashboard.

Expected result: Successful login and redirection.

- **Test Case 2:** Verify that a user with valid credentials for a 'STUDENT' role can successfully log in and is redirected to the student dashboard.

Expected result: Successful login and redirection.

- **Test Case 3:** Attempt to log in with invalid credentials (wrong username/password).

Expected result: Login fails, and a generic error message is displayed.

- **Test Case 4:** Attempt to access protected resource (e.g., /instructor/dashboard) without being authenticated.

Expected result: User is redirected to the login page.

- **Function: View Grades (Student Role)**

- **Test Case 1:** A logged-in student attempts to view their own grades. **Expected result:** The student successfully views their grades.

- **Test Case 2:** A logged-in student attempts to view another student's grades by manipulating the URL or request parameters. **Expected result:** Access is denied, and an appropriate error or redirection occurs (e.g., 403 Forbidden).

- **Function: Post/Update Grades (Instructor Role)**

- **Test Case 1:** A logged-in instructor attempts to post a grade for a student in a course they teach. **Expected result:** The grade is successfully posted/updated.

- **Test Case 2:** A logged-in instructor attempts to post a grade for a student in a course they *do not* teach. **Expected result:** Access is denied, and an appropriate error or redirection occurs.

- **Test Case 3:** A logged-in student attempts to post or update a grade. **Expected result:** Access is denied, and an appropriate error or redirection occurs.

6. Conclusion

This report outlines the comprehensive approach taken to design and implement a Secure Student Information System, emphasizing security throughout the SDLC. By integrating a multi-tier architecture, applying the STRIDE threat model, adhering to robust security design principles, and implementing defensive coding practices, the system aims to provide a secure environment for managing student grades. The detailed security requirements analysis, threat identification, risk assessment, and proposed test cases collectively contribute to building a resilient and trustworthy application. The focus on secure login, input validation, and role-based access control forms the foundation for protecting sensitive academic data against common cyber threats.

7. References

- [1] OWASP Foundation. (n.d.). *OWASP Top 10*. Retrieved from <https://owasp.org/www-project-top-ten/> (Accessed: 2026).
- [2] Microsoft. (n.d.). *Threat Modeling*. Retrieved from [https://learn.microsoft.com/en-us/previous-versions/windows/desktop/dn440717\(v=vs.85\)](https://learn.microsoft.com/en-us/previous-versions/windows/desktop/dn440717(v=vs.85)) (Accessed: 2026).
- [3] Apache Derby Documentation. (n.d.). Retrieved from <https://db.apache.org/derby/docs/> (Accessed: 2026).
- [4] OWASP Top 10 (2021). Open Web Application Security Project. <https://owasp.org/Top10/> (Accessed: 2026).
- [5] MITRE Corporation. *CWE Top 25 Most Dangerous Software Weaknesses*. Available at: [CWE Top 25 Official Page](#) (Accessed: 2026).